



Major Project — End-Term Report

NOESIS

An Operating-System-Inspired Kernel for Autonomous Multi-Agent Intelligence

School of Computer Science · UPES Dehradun · Academic Year 2026–2027

Presented by:

500118979 – Dhruv Shah
500123471 – Manan Nasa

Mentored By:

Dr. Archana Kumari

1	Introduction	Problem · motivation · gap · proposed solution
2	Literature Review	Agent frameworks · OS-for-LLM · capability security · SWOT
3	Objectives	O1–O7 and their status
4	Methodology	Architecture · tech stack · algorithms · process
5	Working Model	Modules · workflow · LLM integration · UI screens
6	Results	Tests · security · determinism · performance · benchmarks
7	Conclusion	Challenges · limitations · future scope
8	References	Key sources

Agent frameworks wire prompts and tools; the runtime concerns are left to the application

Permission boundaries

Any agent can usually call any tool; nothing checks a scoped capability before a side effect (prompt-injection risk).

Reproducibility

Sampling makes the same goal yield different plans — hard to debug, test or audit.

Memory lifecycle

Flat vector stores; promotion, retention and provenance are ad hoc.

Operational visibility

Health, metrics, per-step audit and an operator console are outside the framework.

Problem: apply OS primitives — least-privilege capabilities, deterministic scheduling, tiered memory with provenance, observable syscalls — to multi-agent LLM workflows, usable on one laptop with or without an LLM.

Untrusted output

Prompt injection turns fetched text into instructions → side effects need a check the model cannot bypass.

Engineering discipline

Same input ⇒ same plan enables unit tests, snapshots and exact replay.

Local, free inference

Qwen2.5-Coder 7B via Ollama runs on a laptop; the kernel must treat any LLM as optional.

Explainable design

Syscalls, capability tables, schedulers and interrupts make agent systems teachable and testable.

OS analogy: process → agent · syscall → tool/memory/model call · capability list → permission set · scheduler → step order · interrupt → async event

Literature Review — Existing Gap

Qualitative comparison of system-level concerns

Concern	Agent frameworks	OS-for-LLM research	NOESIS (as built)
Capability check on tool calls	Application-defined	Proposed (AIOS)	Implemented (registry + kernel)
Spawn-time privilege check	Rare	—	Implemented (AgentRoster)
Reproducible planning	Sampling-dependent	Not a focus	Implemented — 50/50 goals bit-identical
Tiered memory + provenance	Vector stores	Context paging (MemGPT)	6 tiers + SHA-256 chain (component)
Works without an LLM	No	No	Yes — deterministic agents
Agent task intelligence	LLM-driven	LLM-driven	Rule-based (limitation)

Gap: no self-hostable runtime in the reviewed work combines kernel-mediated capabilities, bit-exact plan replay, provenance-logged memory and an operator console while staying functional without an LLM.

Strengths

- OS-style, test-covered architecture
- Bit-identical planning (~1 ms/plan)
- Works with or without an LLM
- 310 backend tests pass · 70 % coverage

Weaknesses

- Agents are rule-based heuristics
- Resource routes unauthenticated
- Simulated ingestion / RAG agent
- 6 dashboard pages show placeholders

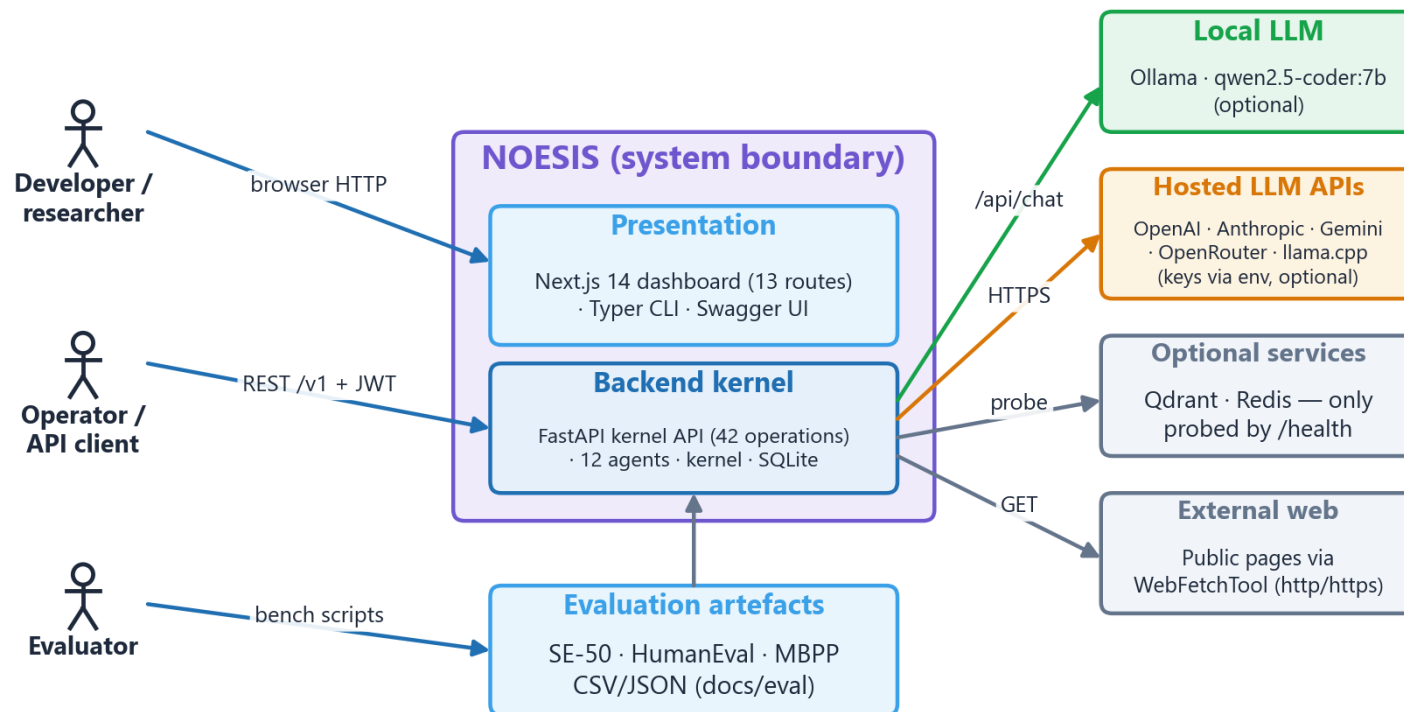
Opportunities

- Wire BM25 + memory bus into runs
- LLM-backed agents with same typed reports
- Functional code benchmarks
- Capability gate on all routes

Threats

- Fast-moving frameworks (LangGraph, AutoGen)
- Local 7B ≈ 23 tok/s — slow
- Prompt injection once LLM drives actions
- Over-claiming in docs erodes trust

Proposed Solution — NOESIS at a Glance



What NOESIS does

Goal → **deterministic plan** (seeded DAG)
Steps run by **12 typed agents**
Each step gets a **least-privilege token**
Every step **audited** (rows, messages, spans)
6-tier memory with SHA-256 provenance
LLM optional — adds rationale only
REST API (42 ops) + Next.js dashboard

Objectives

Objectives

Main objective: a permission-checked, reproducible, auditable agent kernel usable without paid APIs

O1

Agent microkernel: typed agents, capabilities, syscall tracing

Achieved

O2

Deterministic, reproducible planning

Achieved

O3

Least-privilege enforcement (agents, tools, routes)

Partial — routes open

O4

Tiered memory with verifiable provenance

Component only

O5

Optional LLMs with graceful fallback; characterise local model

Achieved

O6

REST API + dashboard with automated tests

Partial — 6 mock pages

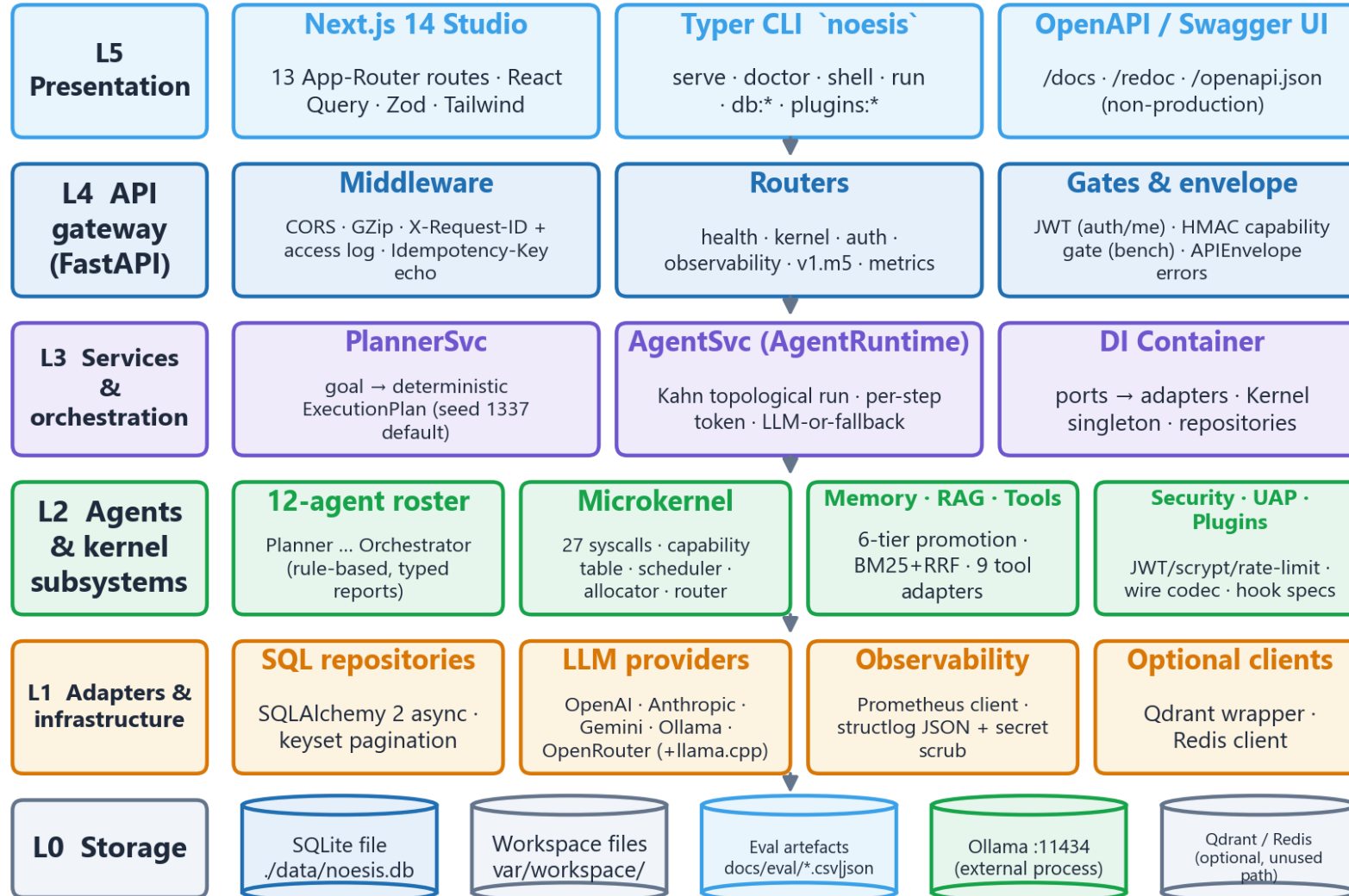
O7

Evaluation harness with statistics

Partial — compile-only

Methodology — System Overview

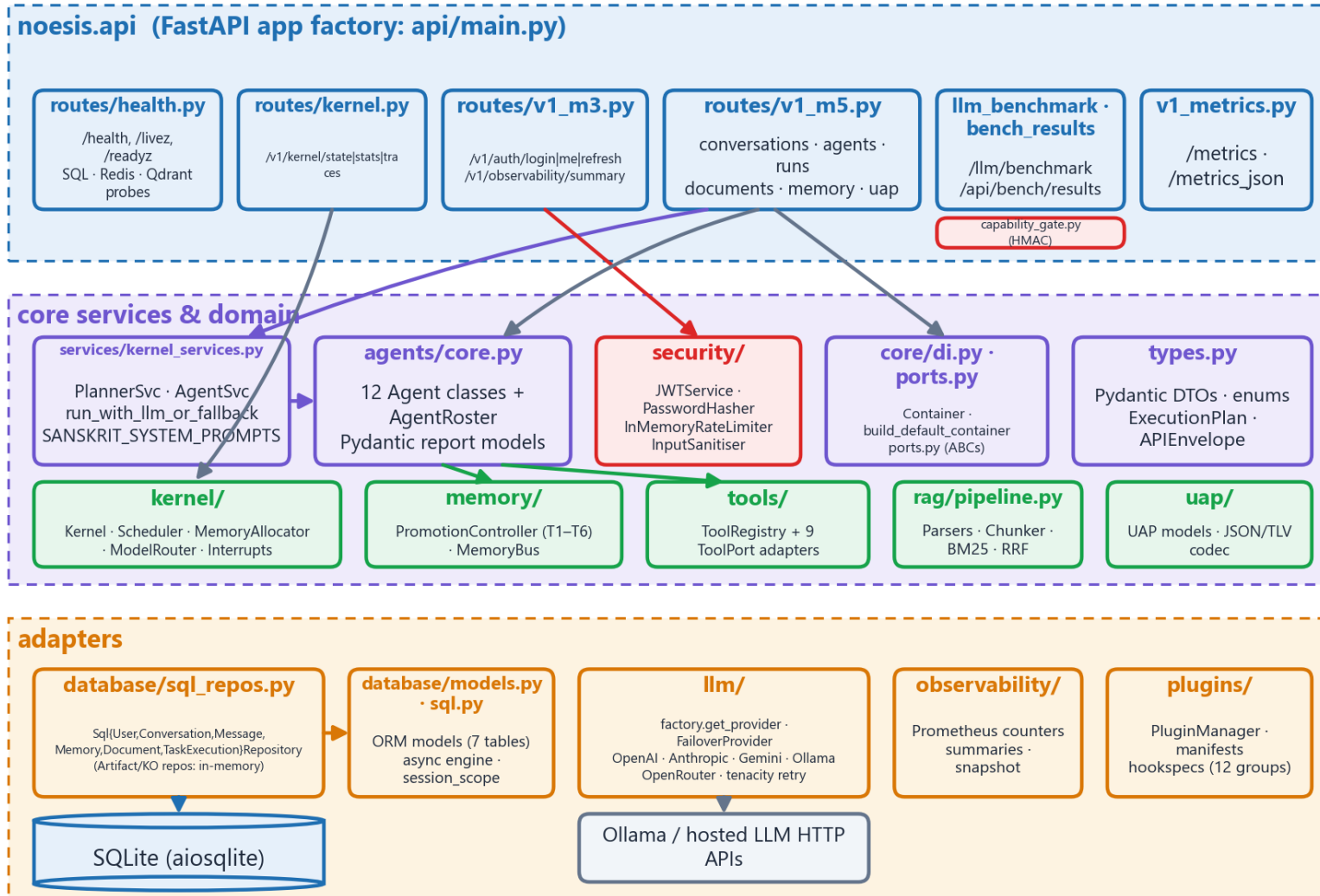
Five layers over storage; dependencies point downwards (ports and adapters)



Methodology — Architecture

Methodology

Backend components and dependencies



Key numbers (measured)

17,072 LOC backend (16 packages)

42 HTTP operations

27 syscall ids · 6 handlers

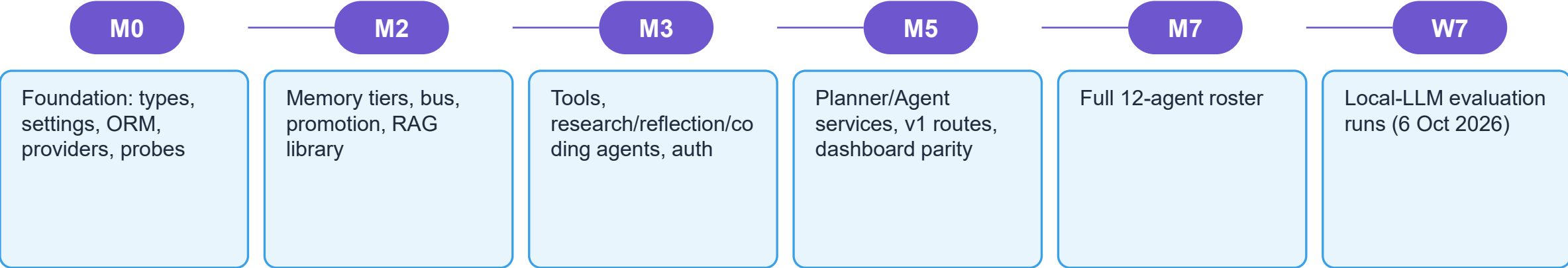
12 agents · 9 tool adapters

5 LLM provider families

7 SQL tables

9,612 lines TS/TSX · 13 routes

Layer	Technology	Why
API	FastAPI · Uvicorn · Pydantic v2	Typed validation, async, OpenAPI
Persistence	SQLAlchemy 2 async · aiosqlite · Alembic	Portable SQL, zero setup
LLM	httpx · tenacity · Ollama (Qwen2.5-Coder 7B)	5 providers, retries, local inference
Security	HS256 JWT · bcrypt/scrypt · HMAC-SHA256	Standard primitives, no external IdP
Observability	structlog · prometheus-client	JSON logs, scrapeable metrics
Frontend	Next.js 14 · React 18 · TS · TanStack Query · Zod · Tailwind	Routing, caching, runtime schemas
Quality	pytest · coverage · Vitest · tsc · ESLint · ruff	Automated gates
Ops	Docker Compose · GitHub Actions (defined)	Reproducible deployment target



Question	Method	Metric
Does it behave as specified?	Run 312 backend + 105 frontend tests (isolated copy)	pass/fail, coverage
Are permissions enforced?	Kernel, roster, tool & HTTP-gate scenarios	expected vs observed
Is planning reproducible?	50 SE-50 goals × 3 runs; 5 seeds	unique SHA-256
How fast?	TestClient latency; /llm/benchmark with Ollama	p50/p95 ms, tok/s
What do benchmarks show?	Re-analyse W7 artefacts	compile-pass, sign-off

Kernel

27 syscall ids, capability table with glob targets, span ring buffer (10 k), scheduler, quota allocator, model router, interrupts

Agents (12)

Planner, Research, Coding, Memory, RAG, Tool, Reflection, Judge, Critic, Executor, Supervisor, Orchestrator — typed reports

Runtime services

PlannerSvc + AgentSvc: Kahn ordering, per-step token (TTL 900 s), LLM-or-fallback, audit rows

Memory

T1 Indriya → T6 Paalak promotion with SHA-256 chain; memory bus with RRF recall

Tools & sandbox

Shell (dry-run), Python AST sandbox, files (rooted), web fetch (http/https) + 5 more adapters

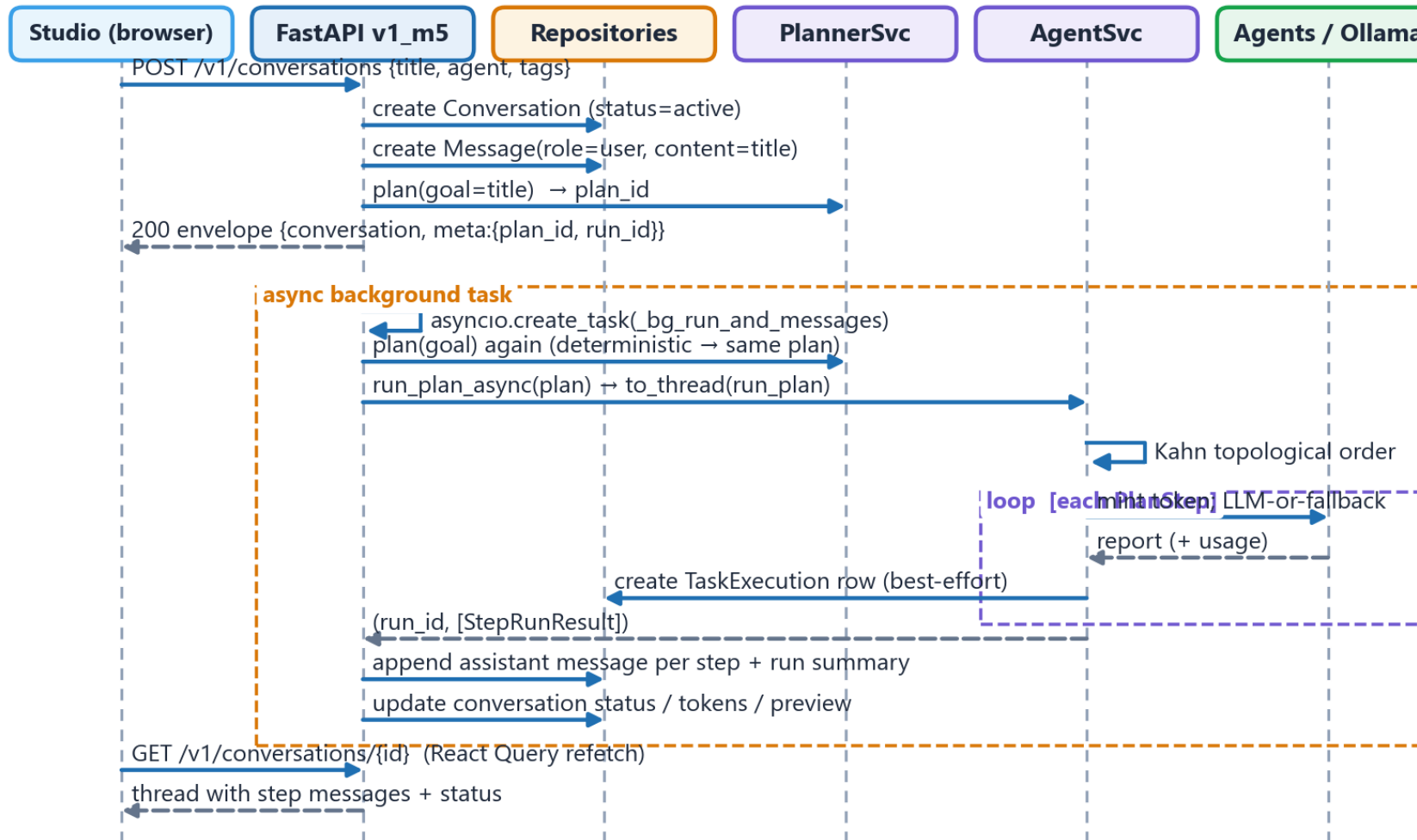
Security & ops

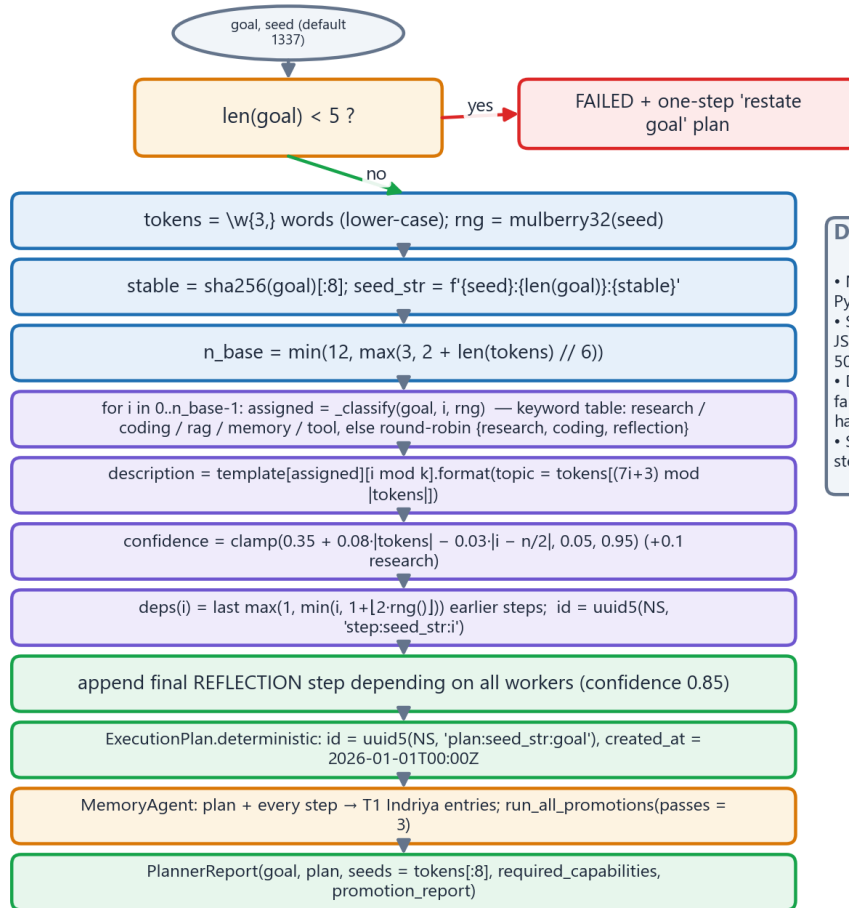
JWT + bcrypt + rate limit; HMAC capability gate; Prometheus; request ids; secret-scrubbed logs

Working Model — End-to-End Workflow

Working Model

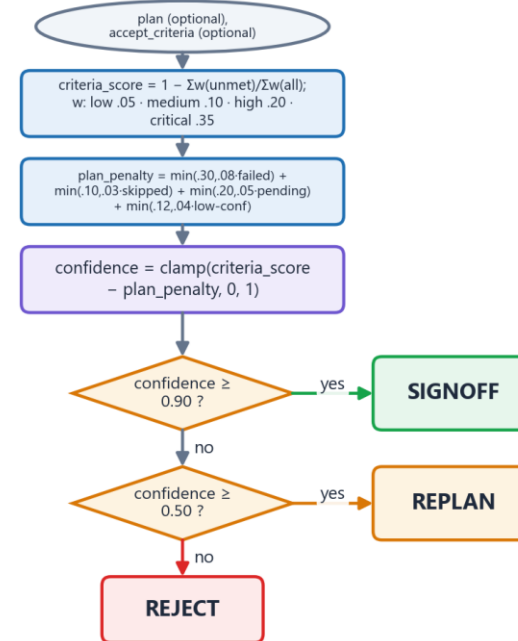
Create conversation → plan → background execution → thread updates





Determinism properties

- No randomness from the wall clock or Python hash() (sha256 used instead).
- Same (goal, seed) ⇒ identical plan JSON ⇒ identical SHA-256 (measured 50/50 SE-50 goals × 3 runs).
- Different seeds change dependency fan-in and ids (5 seeds → 5 distinct hashes).
- Step count measured on SE-50: 4–6 steps (median 6).



Computed grid: 6 steps, 4 high-severity criteria
failed steps →

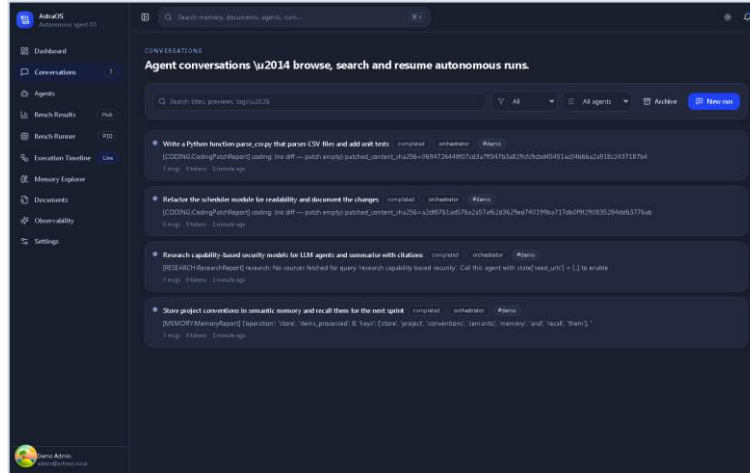
	0	1	2	3
0	SIGNOFF 1.00	SIGNOFF 0.92	REPLAN 0.84	REPLAN 0.76
1	REPLAN 0.75	REPLAN 0.67	REPLAN 0.59	REPLAN 0.51
2	REPLAN 0.50	REJECT 0.42	REJECT 0.34	REJECT 0.26
3	REJECT 0.25	REJECT 0.17	REJECT 0.09	REJECT 0.01

Values computed by ExecutorAgent.evaluate_decision() during this audit (pure function, no sampling).

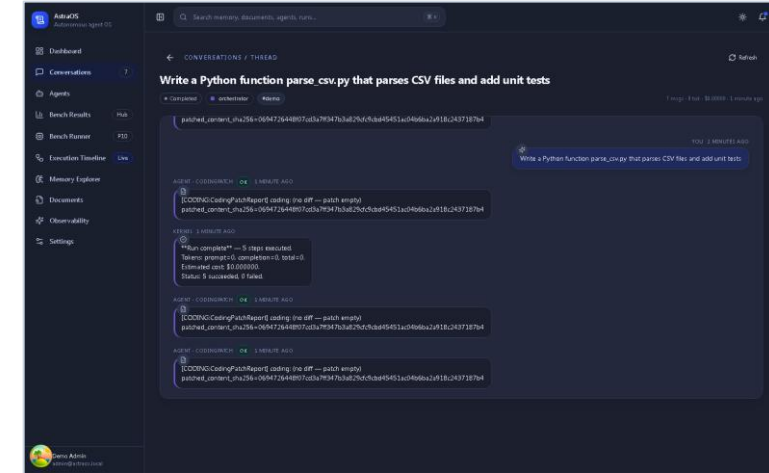
Working Model — Dashboard Screens

Working Model

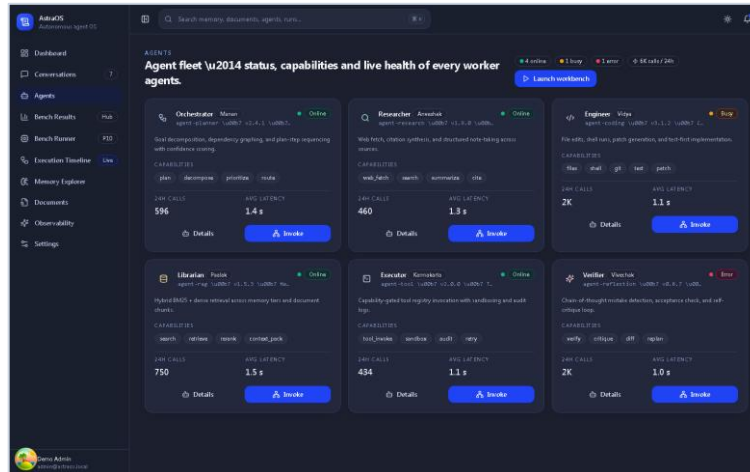
Screenshots of the running application (scratch backend, 6 Oct 2026)



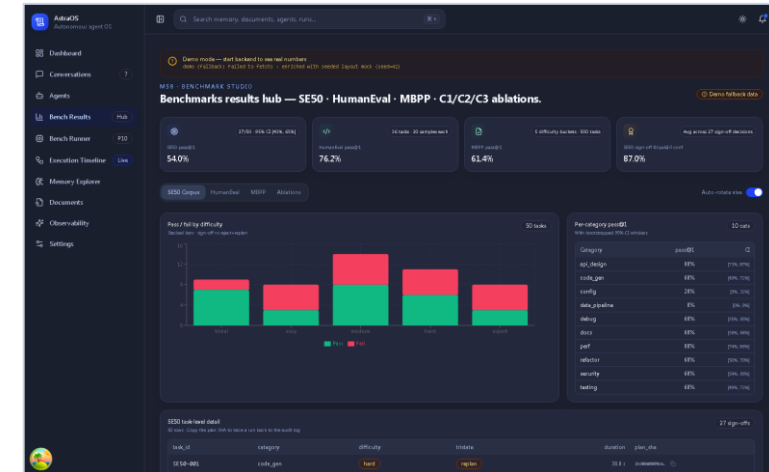
Conversations — live



Thread — live step reports



Agents — placeholder data



Bench Hub — labelled demo data

Results — Functional Testing

Measured on an isolated copy of the working tree (Python 3.13.2, Node 20.18.1)

310/312

backend tests passed (2 xfail)

70.0 %

line coverage

103/105

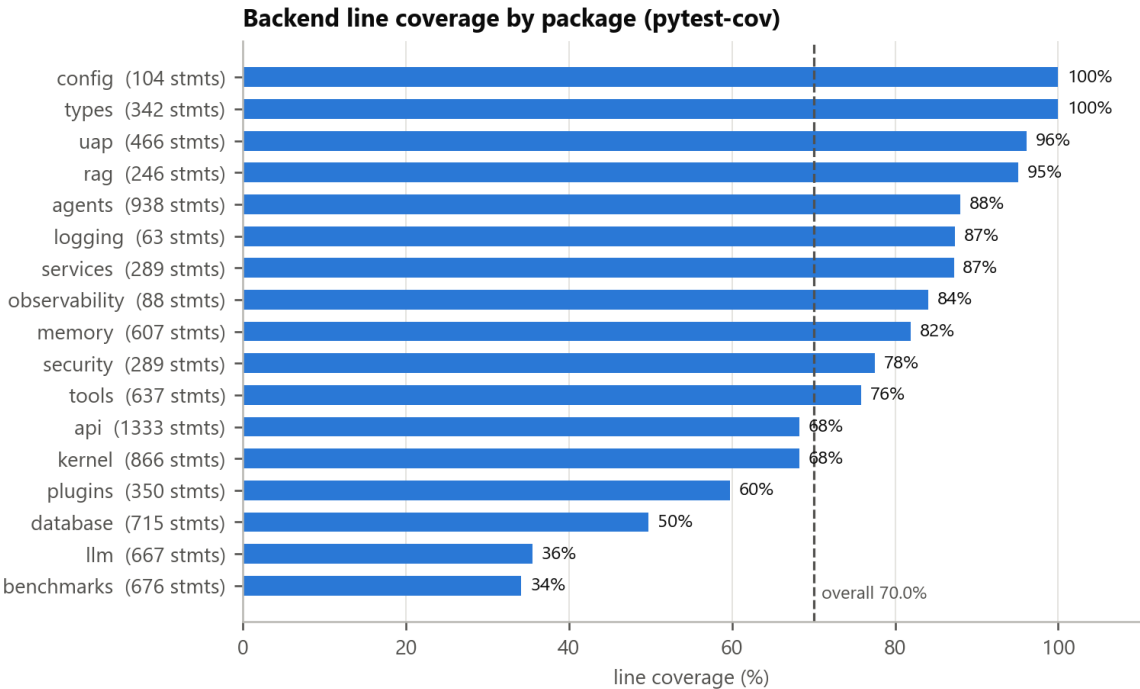
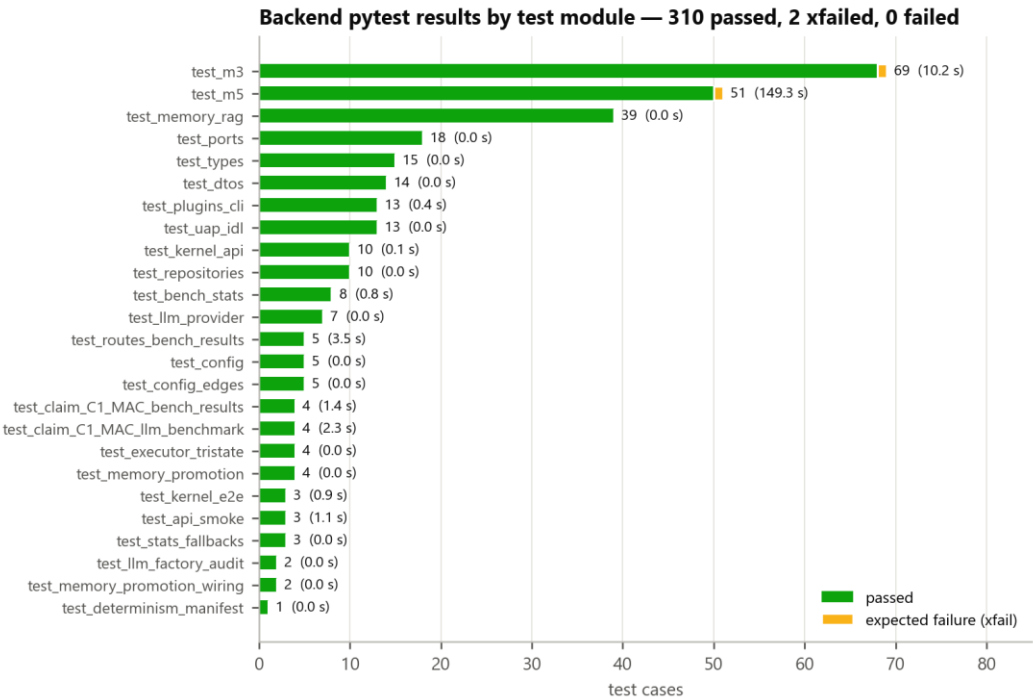
frontend tests passed

0 / 0

tsc errors / ESLint issues

58

API scenarios exercised

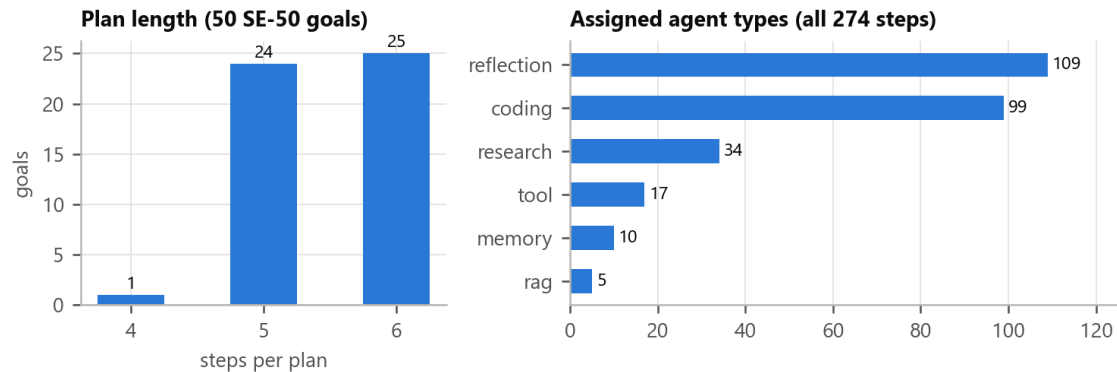


Measured 2026-10-06, Python 3.13.2, Windows 11, isolated copy of the working tree; total wall time 180.2 s (test_m5 dominates because AgentSvc tests call the local LLM).

Measured with coverage.py over noesis/ (cli.py, core/*, llm/_retry.py, llm/parse.py, llm/prompts.py omitted by pyproject configuration).

Scenario (measured)	Result
No capability token	403 C1_DENY_MISSING_CAP
Forged HMAC secret	403 C1_DENY_MAC
Expired token	403 C1_DENY_EXPIRED
Capability not on token	403 C1_DENY_MISSING_CAP
Deny mask llm.*	403 C1_DENY_MASK
Kernel: 0 caps → TOOL_INVOKE	denied
Roster: 12 agents, empty caps	12/12 denied
Sandbox: import os / open()	blocked
Files: ../../etc/passwd	refused (exit 13)
6th login in 60 s	429
GET /v1/conversations, no token	200  open

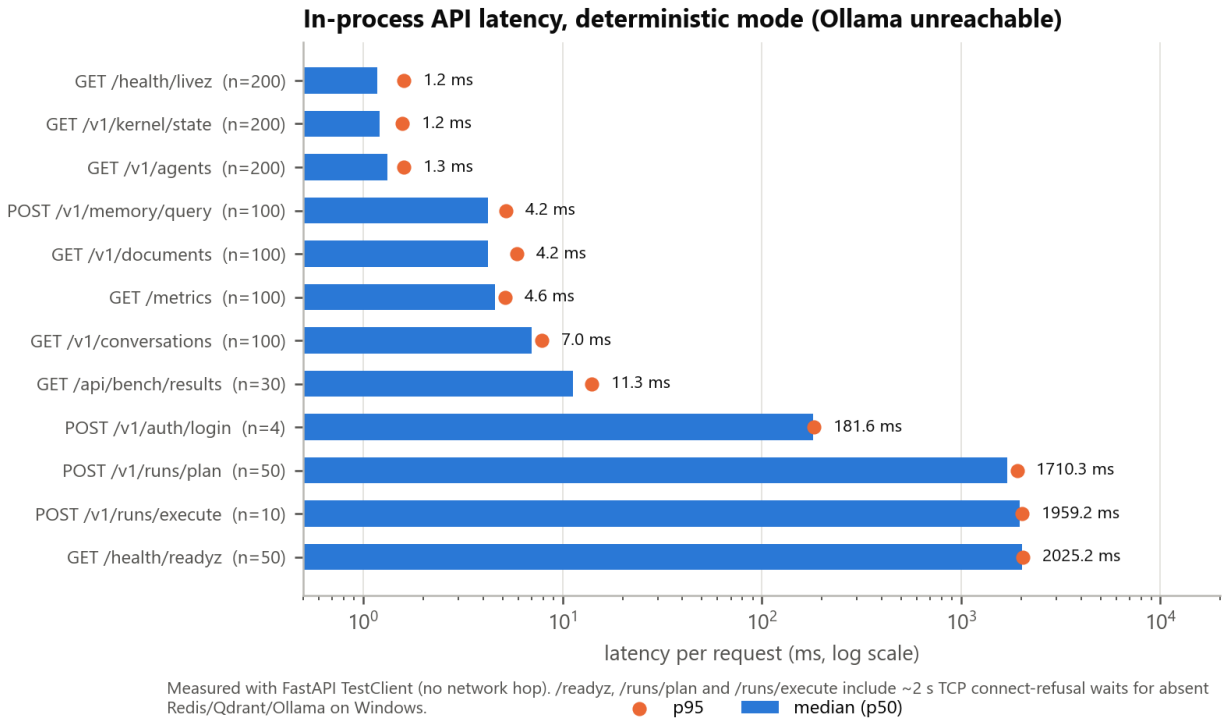
PlannerAgent on SE-50: 50/50 goals hash-identical over 3 runs · median plan time 1.05 ms



Measured 2026-10-06 via PlannerSvc.plan(goal, seed=42) on backend/benchmarks/noesis_se50/corpus.csv.

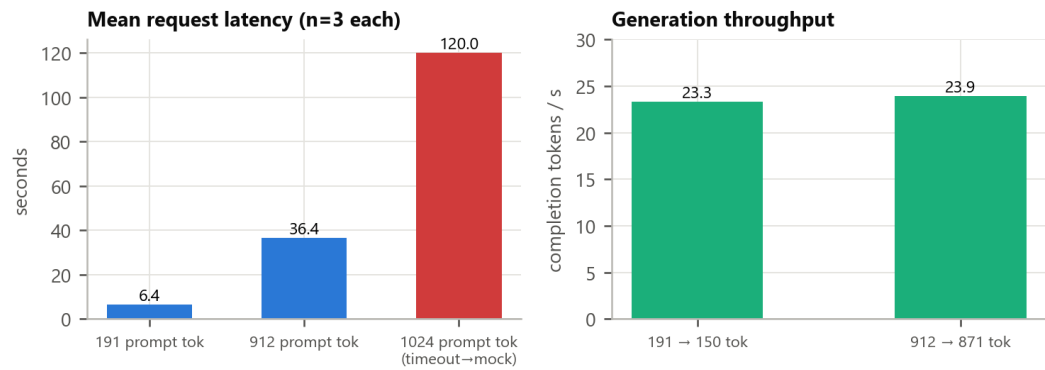
Determinism (measured)

50/50 SE-50 goals → one SHA-256 over 3 runs
Median plan time **1.05 ms**
5 seeds → 5 distinct plans
Promotion chain tail identical in **5/5** repeats



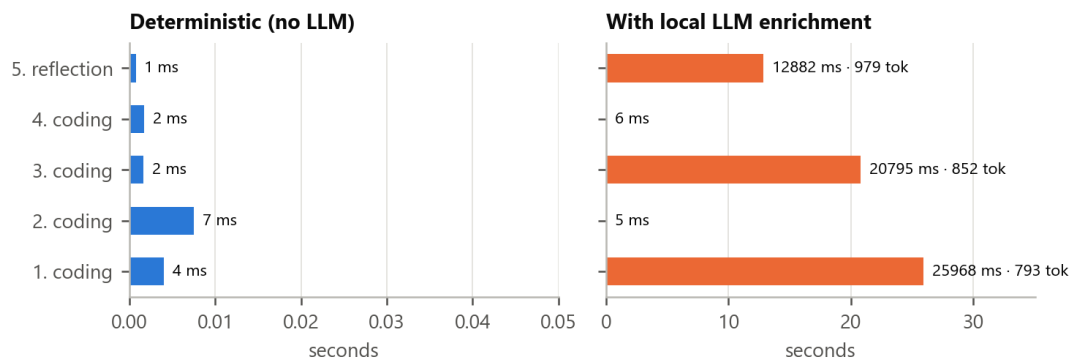
Read endpoints **1–11 ms (p50)** · login **≈ 180 ms** (password hashing) · 5-step run **1.96 s** deterministic vs **60.4 s** with qwen2.5-coder 7B · local LLM **≈ 23 tok/s**

GET /llm/benchmark against local Ollama (qwen2.5-coder:7b-instruct-q4_K_M)



Measured 2026-10-06 on the author's laptop (CPU/GPU not profiled). The 1024-token requests exceeded the 120 s wait_for timeout and the endpoint returned mock values.

POST /v1/runs/execute — per-step duration for the same 5-step plan

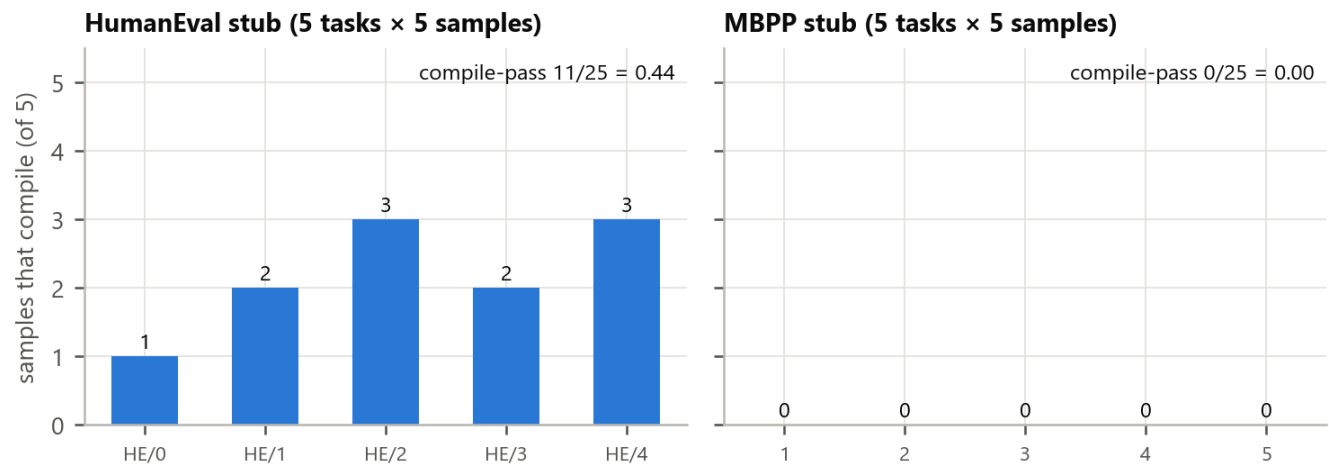


Measured 2026-10-06. Wall time 1.8 s (deterministic, incl. provider probe) vs 60.4 s (LLM). Two LLM-mode steps returned with zero tokens; the cause is not logged.

Results — Benchmark Harness

W7 run of 6 Oct 2026 — reported artefacts, re-analysed: what the numbers do and do not mean

Syntax-compile pass per task — W7 run of 2026-10-06 (qwen2.5-coder 7B, T = 0.7)



Source: docs/eval/w7_weekend_real_Oct06_real. The harness checks compile() only — it does not execute unit tests, so these are NOT functional pass@k scores.

Benchmark	Result	Meaning
SE-50 (5 tasks × 3)	15/15 SIGNOFF	Pipeline integrity; criteria synthesised as met
HumanEval stub (5 × 5)	11/25 compile (0.44)	Syntax only — no tests executed
MBPP stub (5 × 5)	0/25 compile	Samples did not compile
GPT-4o / Claude / Gemini	no artefacts	README figures not reproducible

Honest takeaway

The harness, statistics (pass@k, bootstrap, Wilson, McNemar) and corpus exist and run end-to-end with a local model; meaningful task-quality claims require functional test execution and full corpora — future work.

Level	Tooling	Result
Unit	pytest, pytest-asyncio	310 pass · 2 xfail
Integration (lifespan)	TestClient	3/3
Security claim suites	pytest markers C1/MAC	8/8
Frontend	Vitest + Testing Library	103/105
Static	tsc · ESLint	0 · 0/0
System / audit	58 HTTP scenarios + screenshots	documented
Accessibility	axe-core script	not run

Requirements traceability (65 FRs)

43 — Implemented

7 — Partially implemented

7 — Implemented as library (not wired to API)

6 — Stub / simulated

2 — Planned (not implemented)

Each FR → module → test case → status (SRS §17.2)

14 acceptance test cases executed; 3 fail by design of current code

Contract drift

Backend envelopes vs frontend array checks silently turned live pages into mock pages.

Run identity

run_id stored only in JSON metadata; executions-by-run endpoint returns empty.

Determinism leaks

Python hash() randomisation and wall-clock time had to be removed from plan identity.

Slow local inference

≈ 23 tok/s; 1 024-token prompts exceed the 120 s timeout.

Windows TCP refusal

~2 s per probe to absent services dominated plan/execute latency.

Evidence vs claims

README numbers (tests, routes, benchmarks) had to be reconciled with the code.

- L1 Agents are rule-based; the LLM only adds rationale
- L2 Resource routes have no authentication/authorisation
- L3 Document upload & RAG agent simulated (BM25 library not wired)
- L4 Memory promotion not driven by live runs; persistence not wired
- L5 Six dashboard pages show placeholder data
- L6 Benchmarks: compile-only and synthesised sign-off
- L7 Single-process state; containers & CI not verified here
- L8 Two frontend tests fail on the working tree

Secure the API

require_auth + ownership on all /v1 routes; reject default HMAC secret; SSRF filter; process sandbox

Smarter agents

LLM-backed agents emitting the same typed reports (structured output) with deterministic fallback

Real retrieval & memory

Wire BM25/RRF + embeddings/Qdrant into upload and RAGAgent; drive promotion from recall; persist tiers

Live dashboard

Unwrap envelopes; stream step progress (SSE); run_id column

Rigorous evaluation

Functional pass@k on full HumanEval/MBPP; SWE-bench Lite; paired McNemar comparisons

Scale & edge

Externalise state; verified containers; characterise Jetson-class hardware

Justification of objectives

Objective	Status	Evidence
O1 Agent microkernel	Achieved	Kernel tests, 9 scenarios, 12-agent roster
O2 Deterministic planning	Achieved	50/50 goals, ~1 ms/plan
O3 Least privilege	Partial	Agents/tools/gate enforced; resource routes open
O4 Tiered memory + provenance	Component	Controlled scenario, 5/5 identical chains
O5 Optional LLM	Achieved	Fallback; 23–24 tok/s measured
O6 API + dashboard	Partial	42 ops, live conversations, 6 placeholder pages
O7 Evaluation harness	Partial	Runs end-to-end; compile-only

NOESIS is a reproducible, auditable, permission-checked kernel substrate for agent workflows. Its agent intelligence — not its kernel — is the main subject of future work.

Selected; full list (47 entries) in the SRS and report

- [9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in Proc. Int. Conf. Learning Representations (ICLR), 2023, arXiv:2210.03629.
- [13] Q. Wu et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023.
- [14] S. Hong et al., "MetaGPT: Meta programming for a multi-agent collaborative framework," in Proc. Int. Conf. Learning Representations (ICLR), 2024, arXiv:2308.00352.
- [7] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," arXiv:2310.08560, 2023.
- [8] K. Mei, Z. Li, S. Xu, R. Ye, Y. Ge, and Y. Zhang, "AIOS: LLM agent operating system," arXiv:2403.16971, 2024.
- [2] J. B. Dennis and E. C. Van Horn, "Programming semantics for multiprogrammed computations," Communications of the ACM, vol. 9, no. 3, pp. 143–155, 1966, doi: 10.1145/365230.365252.
- [3] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," Proceedings of the IEEE, vol. 63, no. 9, pp. 1278–1308, 1975, doi: 10.1109/PROC.1975.9939.
- [4] M. S. Miller, K.-P. Yee, and J. Shapiro, "Capability myths demolished," Systems Research Laboratory, Johns Hopkins University, Tech. Rep. SRL2003-02, 2003.
- [1] R. C. Atkinson and R. M. Shiffrin, "Human memory: A proposed system and its control processes," in The Psychology of Learning and Motivation, vol. 2, K. W. Spence and J. T. Spence, Eds. New York: Academic Press, 1968, pp. 89–195, doi: 10.1016/S0079-7421(08)60422-3.
- [20] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009, doi: 10.1561/15000000019.
- [21] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in Proc. 32nd Int. ACM SIGIR Conf., 2009, pp. 758–759, doi: 10.1145/1571941.1572114.
- [23] M. Chen et al., "Evaluating large language models trained on code," arXiv:2107.03374, 2021.
- [24] J. Austin et al., "Program synthesis with large language models," arXiv:2108.07732, 2021.
- [26] B. Hui et al., "Qwen2.5-Coder technical report," arXiv:2409.12186, 2024.
- [27] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec), 2023, arXiv:2302.12173.



Thank You

Questions?

Documentation package: SRS v1.0 · Major Project Report · this deck · evidence (tests, measurements, screenshots)